

Proyecto: Casa de la Impresión — Sistema de Pedidos por Microservicios

Nombre: Sebastián Valderrama Romero
Asignatura: DSY1103 — Desarrollo FullStack 1
Evaluación Parcial N°3

Documento de análisis previo al desarrollo
Repositorio: github.com/sevalderramar/Casa-de-la-Impresion

1. Identificación del problema

1.1 Contexto del dominio

Casa de la Impresión es una imprenta orientada a emprendedores y artesanos que transforma ideas en productos físicos (etiquetas, adhesivos y pendones, entre otros) impresos a pedido. Como en muchos negocios de manufactura por encargo, cada venta involucra varias etapas que hoy se gestionan de forma fragmentada: el registro del cliente y su pedido, la verificación de stock de los insumos o productos solicitados, el seguimiento del avance de fabricación, el cambio de estado del pedido a medida que progresa y, finalmente, su despacho a través de un transportista.

Cuando esta información se administra en planillas, cuadernos o conversaciones sueltas con el cliente, el encargado de pedidos pierde visibilidad del estado real de cada trabajo: no es posible saber con certeza si un pedido sigue en cola, está en producción o ya fue despachado sin llamar manualmente a cada área. Esto genera reprocesos, respuestas tardías a los clientes y dificulta calcular indicadores básicos del negocio, como qué productos se venden más o quiénes son los clientes más frecuentes.

El dominio que aborda este proyecto se acota deliberadamente a la gestión del ciclo de vida de un pedido de impresión: desde que el cliente lo solicita, pasando por la validación de catálogo y stock, la fabricación, el cambio de estado y el despacho, hasta la generación de métricas de venta. No se busca resolver la operación completa de la imprenta (por ejemplo, no se aborda facturación ni diseño gráfico de los productos), sino dar trazabilidad de extremo a extremo al flujo de pedidos.

1.2 Declaración del problema

Casa de la Impresión carece de un sistema centralizado que permita registrar pedidos, validar clientes y productos, dar seguimiento al estado de fabricación y despacho, y medir el desempeño comercial, lo que provoca pérdida de trazabilidad y demoras en la atención de los clientes.

1.3 Actores / usuarios involucrados

- **Administrador del sistema:** gestiona usuarios internos, sus roles y credenciales de acceso (auth-service).
- **Encargado de pedidos:** registra clientes, crea pedidos, valida productos y gestiona el cambio de estado de cada pedido.
- **Operario de fabricación:** actualiza el avance de las órdenes de fabricación asociadas a cada pedido.
- **Encargado de despacho:** registra el despacho de un pedido, asigna transportista y código de seguimiento.
- **Transportista:** entidad consultada para asignar la cobertura y datos de contacto de los despachos.
- **Cliente:** entidad sobre la que se registran y consultan los pedidos realizados.
- **Analista / dirección comercial:** consume las métricas de ventas, ranking de clientes y productos más vendidos (metrica-service).

2. Requerimientos del sistema

2.1 Requerimientos funcionales

- RF-01** — Registrar un nuevo cliente con nombre, RUT, email, teléfono, dirección, comuna y región.
- RF-02** — Listar todos los clientes registrados en el sistema.
- RF-03** — Consultar un cliente por su ID o por su RUT.
- RF-04** — Actualizar los datos de un cliente existente.
- RF-05** — Eliminar un cliente del sistema.
- RF-06** — Registrar un nuevo producto del catálogo con nombre, descripción, categoría, precio y stock.
- RF-07** — Listar todos los productos del catálogo y filtrarlos por categoría o por nombre.
- RF-08** — Actualizar y eliminar un producto existente del catálogo.
- RF-09** — Crear un pedido asociado a un cliente existente, con uno o más ítems de producto y su cantidad.
- RF-10** — Validar contra cliente-service y producto-service que el cliente y los productos del pedido existan antes de confirmar la creación.
- RF-11** — Listar todos los pedidos y consultar un pedido específico por su número.
- RF-12** — Listar los pedidos asociados a un cliente determinado.
- RF-13** — Cambiar el estado de un pedido (por ejemplo, de COLA a PRODUCCIÓN) y registrar el cambio en estado-service.
- RF-14** — Consultar el historial completo de cambios de estado de un pedido.
- RF-15** — Eliminar un pedido existente.
- RF-16** — Crear una orden de fabricación asociada a un pedido existente, validando dicho pedido contra pedido-service.
- RF-17** — Consultar una orden de fabricación por su ID y actualizar su estado, registrando el historial de cambios.
- RF-18** — Registrar un despacho asociado a un pedido existente, indicando tipo de despacho, transportista y código de seguimiento.
- RF-19** — Listar despachos (con filtro opcional por tipo) y consultar el despacho asociado a un número de pedido.
- RF-20** — Actualizar los datos de un despacho existente.
- RF-21** — Registrar un nuevo transportista con nombre, código interno, contacto y regiones de cobertura.
- RF-22** — Listar los transportistas activos, consultar uno por ID y actualizar sus datos.
- RF-23** — Autenticar a un usuario interno (login) y emitir un token JWT válido para operar en el sistema.
- RF-24** — Registrar, listar y actualizar usuarios internos del sistema, asignando un rol (solo administrador).
- RF-25** — Calcular y exponer métricas por cliente (monto total, cantidad de pedidos, frecuencia anual) y un ranking de clientes.

RF-26 — Calcular y exponer el ranking de productos más vendidos y un resumen de ventas en un rango de fechas.

RF-27 — Registrar una entrada de log por cada operación relevante ejecutada en cualquier microservicio y permitir su consulta filtrada por servicio y fecha.

2.2 Requerimientos no funcionales

RNF-01 — Cada microservicio debe responder con el código HTTP adecuado a cada escenario (200, 201, 204, 400, 404, 409 o 503), manejado de forma centralizada mediante un `GlobalExceptionHandler` propio por servicio.

RNF-02 — Toda solicitud de creación o actualización debe validar los datos de entrada mediante anotaciones de Bean Validation (`@Valid`) antes de procesarse, retornando 400 si los datos son inválidos.

RNF-03 — Cada microservicio debe mantener su propia base de datos (perfil H2 independiente por servicio), sin acceso directo a las tablas de otro servicio; toda comunicación entre dominios debe hacerse vía REST/OpenFeign.

RNF-04 — Las rutas de cada microservicio deben seguir convenciones REST consistentes (sustantivo en plural bajo `/api/`, verbos HTTP correctos para cada operación).

RNF-05 — Todo acceso a los endpoints protegidos del sistema debe requerir un token JWT válido emitido por `auth-service`, propagado entre servicios mediante un interceptor Feign común (`FeignJwtInterceptor`).

RNF-06 — Las operaciones relevantes del sistema deben quedar trazadas mediante registros centralizados en `log-service`, permitiendo auditar qué servicio ejecutó qué operación y con qué resultado.

RNF-07 — Cada endpoint debe retornar una respuesta en menos de 500 ms bajo condiciones normales de operación local.

RNF-08 — Los servicios principales del flujo de pedidos (`api-gateway`, `cliente-service`, `producto-service`, `pedido-service`, `estado-service`) deben poder desplegarse y orquestarse mediante Docker Compose de forma independiente entre sí.

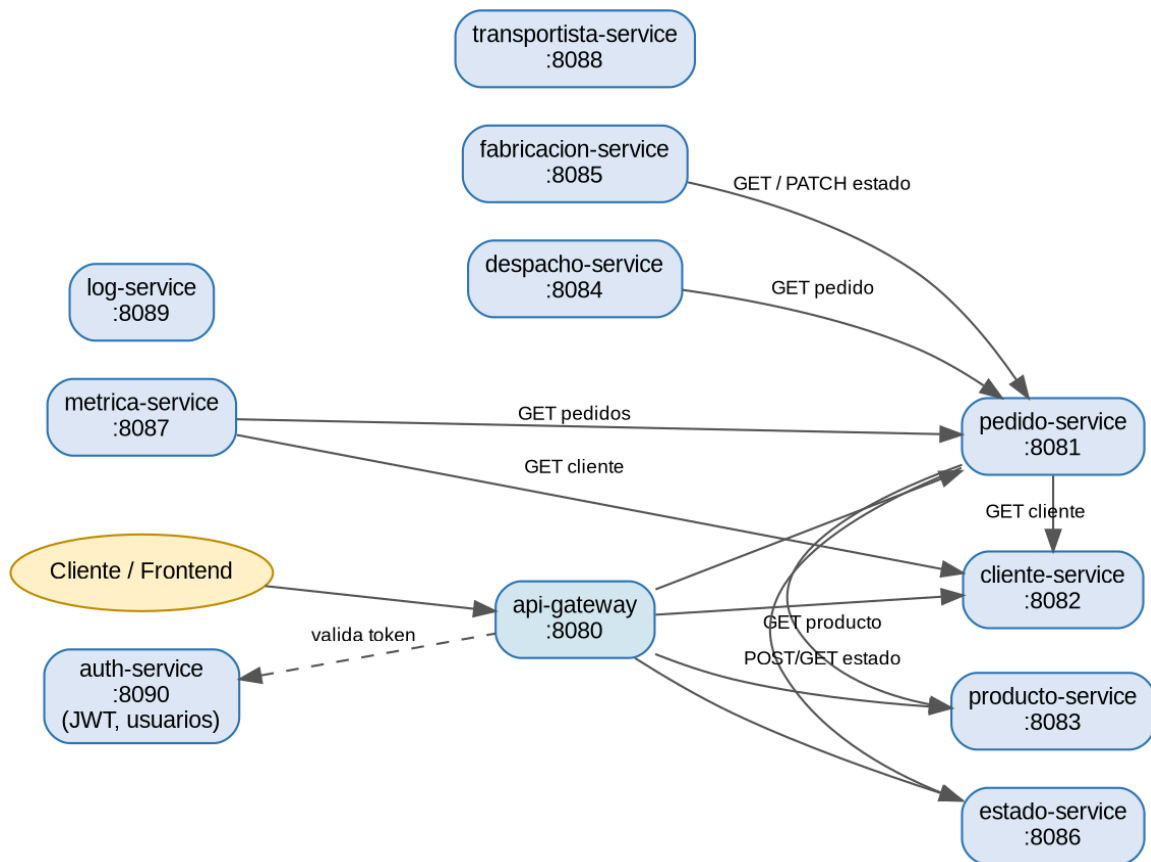
3. Solución propuesta

La solución implementada divide el dominio de gestión de pedidos en diez microservicios independientes desarrollados en Spring Boot (Java 21), cada uno con responsabilidad única, base de datos propia y expuestos a través de un único punto de entrada: un api-gateway (Spring Cloud Gateway) que enruta las peticiones del cliente hacia el servicio correspondiente. La autenticación se resuelve de forma transversal mediante auth-service, que emite tokens JWT validados por un filtro compartido (JwtAuthFilter) presente en cada microservicio, y propagados entre servicios mediante un interceptor Feign común.

El flujo principal del negocio gira en torno a pedido-service, que actúa como orquestador: al crear un pedido valida la existencia del cliente contra cliente-service y de los productos contra producto-service, y registra cada cambio de estado en estado-service. A partir de un pedido válido, fabricacion-service gestiona su avance productivo y despacho-service gestiona su entrega, ambos consultando a pedido-service vía Feign para confirmar que el pedido referenciado existe. Finalmente, metrica-service consulta pedido-service y cliente-service para construir reportes de ventas y rankings, mientras que log-service y transportista-service operan como servicios de soporte transversal.

Una arquitectura de microservicios es apropiada para este dominio porque cada etapa del ciclo de vida del pedido (catálogo, cliente, fabricación, estado, despacho, métricas) tiene un ritmo de cambio y una responsabilidad distinta: el catálogo de productos cambia con baja frecuencia, mientras que el estado de los pedidos cambia constantemente; separar estos dominios permite escalar, desplegar y mantener cada uno de forma independiente, evitando que una falla o un cambio en un módulo (por ejemplo, métricas) afecte la disponibilidad del flujo crítico de creación de pedidos. La comunicación entre servicios se implementa mediante clientes OpenFeign declarativos, lo que mantiene bajo acoplamiento y contratos REST explícitos entre dominios.

3.1 Diagrama conceptual de la arquitectura



El diagrama muestra el api-gateway como punto de entrada único hacia cliente-service, producto-service, pedido-service y estado-service (servicios incluidos en la demo Docker), la validación de tokens contra auth-service, y las dependencias internas vía Feign de despacho-service, fabricacion-service y metrica-service hacia pedido-service (y de este último hacia cliente-service, producto-service y estado-service).

4. Casos de uso / historias de usuario

Historia 1

Como encargado de pedidos, quiero registrar un nuevo cliente con su RUT y datos de contacto, para poder asociarlo a futuros pedidos sin volver a pedir sus datos.

Criterios de aceptación:

- Dado un RUT no registrado, cuando se envía POST /api/clientes con datos válidos, entonces se crea el cliente y se retorna 201 con su ID.
- El sistema debe retornar 409 si el RUT o el email ya están registrados.
- El sistema debe retornar 400 si falta un campo obligatorio.

Historia 2

Como encargado de pedidos, quiero crear un pedido seleccionando un cliente y uno o más productos, para iniciar el proceso de fabricación y despacho de forma trazable.

Criterios de aceptación:

- Dado un clienteld y una lista de items válidos, cuando se envía POST /api/pedidos, entonces se crea el pedido en estado COLA y se retorna 201 con su numeroPedido.
- El sistema debe retornar 404 si el cliente o algún producto no existen.
- El sistema debe retornar 503 si cliente-service o producto-service no están disponibles.

Historia 3

Como encargado de pedidos, quiero consultar todos los pedidos realizados por un cliente, para responder rápidamente cuando el cliente pregunta por el estado de sus encargos.

Criterios de aceptación:

- Dado un clienteld existente, cuando se envía GET /api/pedidos/cliente/{clienteld}, entonces se retorna 200 con la lista de pedidos del cliente.
- El sistema debe retornar una lista vacía si el cliente no tiene pedidos.

Historia 4

Como encargado de pedidos, quiero cambiar el estado de un pedido a medida que avanza, para mantener informado al cliente y al equipo del progreso real del encargo.

Criterios de aceptación:

- Dado un numeroPedido existente, cuando se envía POST /api/pedidos/{numeroPedido}/estado con un nuevo estado válido, entonces se actualiza el pedido y se registra el cambio en estado-service.
- El sistema debe retornar 400 si el estado enviado es inválido.
- La respuesta debe incluir el estado actualizado del pedido.

Historia 5

Como encargado de pedidos, quiero consultar el historial de cambios de estado de un pedido, para auditar cuánto tiempo permaneció el pedido en cada etapa.

Criterios de aceptación:

- Dado un numeroPedido existente, cuando se envía GET /api/pedidos/{numeroPedido}/historial, entonces se retorna 200 con la lista cronológica de cambios de estado.
- El sistema debe retornar 404 si el pedido no tiene historial registrado.

Historia 6

Como operario de fabricación, quiero crear una orden de fabricación a partir de un pedido confirmado, para iniciar formalmente la producción del encargo.

Criterios de aceptación:

- Dado un pedidoId existente, cuando se envía POST /api/fabricacion, entonces se crea la orden de fabricación y se retorna 201.
- El sistema debe retornar 404 si el pedido referenciado no existe en pedido-service.
- El sistema debe retornar 409 si ya existe una orden de fabricación para ese pedido.

Historia 7

Como operario de fabricación, quiero actualizar el estado de una orden de fabricación, para reflejar el avance real del trabajo de impresión y dejar registro del cambio.

Criterios de aceptación:

- Dado un id de orden existente, cuando se envía PATCH /api/fabricacion/{id}/estado, entonces se actualiza el estado y se agrega una entrada al historial de fabricación.
- El sistema debe retornar 404 si la orden no existe.
- El sistema debe retornar 409 si la transición de estado solicitada no es válida.

Historia 8

Como encargado de despacho, quiero registrar el despacho de un pedido con su transportista y código de seguimiento, para que el cliente pueda hacer seguimiento de su encargo hasta la entrega.

Criterios de aceptación:

- Dado un numeroPedido existente, cuando se envía POST /api/despachos, entonces se crea el despacho y se retorna 201 con su ID.
- El sistema debe retornar 404 si el pedido no existe en pedido-service.
- El sistema debe retornar 409 si ya existe un despacho para ese número de pedido.

Historia 9

Como analista comercial, quiero consultar el ranking de clientes por monto total comprado, para identificar a los clientes más relevantes para el negocio.

Criterios de aceptación:

- Dado que existen pedidos registrados, cuando se envía GET /api/metricas/clientes/ranking, entonces se retorna 200 con la lista de clientes ordenada por monto total descendente.
- El parámetro limite debe acotar la cantidad de resultados retornados (por defecto 10).

Historia 10

Como analista comercial, quiero consultar el resumen de ventas en un rango de fechas, para evaluar el desempeño comercial de un período determinado.

Criterios de aceptación:

- Dado un rango de fechas desde/hasta, cuando se envía GET /api/metricas/ventas, entonces se retorna 200 con el resumen de ventas del período.
- El sistema debe retornar 400 si la fecha 'desde' es posterior a la fecha 'hasta'.

Historia 11

Como administrador, quiero iniciar sesión con su correo y contraseña, para obtener un token JWT que le permita operar en los demás microservicios.

Criterios de aceptación:

- Dadas credenciales válidas, cuando se envía POST /api/auth/login, entonces se retorna 200 con el token JWT y los datos básicos del usuario.
- El sistema debe retornar 401 si las credenciales son inválidas o el usuario está inactivo.

Historia 12

Como administrador, quiero registrar un nuevo usuario interno y asignarle un rol, para controlar quién puede operar el sistema y con qué nivel de acceso.

Criterios de aceptación:

- Dado un correo no registrado, cuando se envía POST /api/auth/usuarios, entonces se crea el usuario y se retorna 201.
- El sistema debe retornar 409 si el correo ya está en uso.
- Solo un usuario con rol ADMIN puede invocar este endpoint.

5. Definición de microservicios

A continuación se detallan los diez microservicios de dominio implementados, además del api-gateway como componente de enrutamiento (sin lógica de negocio propia, por lo que no se incluye con la plantilla de entidades/endpoints).

auth-service (puerto 8090)

Responsabilidad: Gestiona la autenticación de usuarios internos del sistema y la emisión/validación de tokens JWT que protegen al resto de los microservicios.

Entidades JPA:

- Usuario: id, nombre, email, password (hash BCrypt), rol (enum: ADMIN, ENCARGADO_PEDIDOS, etc.), activo.

Endpoints REST:

- **POST** /api/auth/login — autentica un usuario y retorna un token JWT.
- **POST** /api/auth/logout — confirma el cierre de sesión (invalidación del lado del cliente).
- **GET** /api/auth/ping — healthcheck público del servicio.
- **GET** /api/auth/usuarios — lista los usuarios del sistema (solo ADMIN).
- **POST** /api/auth/usuarios — crea un nuevo usuario (solo ADMIN).
- **PUT** /api/auth/usuarios/{id} — actualiza un usuario existente (solo ADMIN).

Comunica con: No consume otros microservicios; en cambio, todos los demás servicios validan el JWT emitido aquí mediante un filtro compartido (JwtAuthFilter) y un interceptor Feign (FeignJwtInterceptor) que propaga el token entre llamadas internas.

Base de datos: Base H2 propia (perfil h2), esquema de usuarios independiente del resto de los servicios.

cliente-service (puerto 8082)

Responsabilidad: Administra el ciclo de vida de los clientes que realizan pedidos en la imprenta.

Entidades JPA:

- Cliente: id, nombre, rut (único), email (único), telefono, direccion, comuna, region, fechaRegistro.

Endpoints REST:

- **POST** /api/clientes — registra un nuevo cliente.
- **GET** /api/clientes — lista todos los clientes registrados.
- **GET** /api/clientes/{id} — obtiene un cliente por id.
- **GET** /api/clientes/rut/{rut} — busca un cliente por RUT.
- **PUT** /api/clientes/{id} — actualiza los datos de un cliente.
- **DELETE** /api/clientes/{id} — elimina un cliente.

Comunica con: Es consumido vía Feign por pedido-service (para validar al cliente de un pedido) y por metrica-service (para enriquecer rankings de clientes).

Base de datos: Base H2 propia, tabla clientes con restricciones de unicidad en RUT y email.

producto-service (puerto 8083)

Responsabilidad: Mantiene el catálogo de productos de impresión (etiquetas, adhesivos, pendones, resmas, etc.) junto a su stock y precio.

Entidades JPA:

- Producto: id, nombre (único), descripcion, categoria, precio, stock, fechaCreacion.

Endpoints REST:

- **POST** /api/productos — crea un nuevo producto en el catálogo.
- **GET** /api/productos — lista todos los productos.
- **GET** /api/productos/{id} — obtiene un producto por id.
- **GET** /api/productos/nombre/{nombre} — busca un producto por nombre exacto.
- **GET** /api/productos/categoria/{categoria} — filtra productos por categoría.
- **PUT** /api/productos/{id} — actualiza un producto existente.
- **DELETE** /api/productos/{id} — elimina un producto del catálogo.

Comunica con: Es consumido vía Feign por pedido-service para validar el producto y su precio al momento de generar un pedido.

Base de datos: Base H2 propia, tabla productos con restricción de unicidad en el nombre.

pedido-service (puerto 8081)

Responsabilidad: Orquesta la creación y el seguimiento de los pedidos, validando cliente y productos contra otros servicios y registrando el detalle de ítems.

Entidades JPA:

- Pedido: numeroPedido, clienteId, estado, tipoDespacho, monto, fechaCreacion; relación @OneToMany con ItemPedido.
- ItemPedido: id, productId, nombreProducto, cantidad, precioUnitario, subtotal; relación @ManyToOne con Pedido.

Endpoints REST:

- **POST** /api/pedidos — crea un pedido validando cliente, productos y estado inicial.
- **GET** /api/pedidos — lista todos los pedidos.
- **GET** /api/pedidos/{numeroPedido} — obtiene un pedido por número.
- **GET** /api/pedidos/numero/{numeroPedido} — obtiene un pedido recibiendo el número como texto.
- **GET** /api/pedidos/cliente/{clienteId} — lista los pedidos de un cliente.
- **POST** /api/pedidos/{numeroPedido}/estado — cambia el estado de un pedido.

- **GET** /api/pedidos/{numeroPedido}/historial — consulta el historial de cambios de estado de un pedido.
- **DELETE** /api/pedidos/{numeroPedido} — elimina un pedido.

Comunica con: Servicio core del dominio: llama vía Feign a cliente-service (valida el cliente), producto-service (valida productos y precios) y estado-service (registra y consulta cambios de estado). Es consumido a su vez por despacho-service, fabricacion-service y metrica-service.

Base de datos: Base H2 propia, tablas pedidos e items_pedido.

estado-service (puerto 8086)

Responsabilidad: Registra y consulta el historial de cambios de estado de los pedidos (trazabilidad del flujo COLA → PRODUCCIÓN → DESPACHO, etc.).

Entidades JPA:

- CambioEstado: id, numeroPedido, estadoAnterior, estadoNuevo, fechaCambio, observacion.

Endpoints REST:

- **POST** /api/estados — registra un nuevo cambio de estado para un pedido.
- **GET** /api/estados/pedido/{numeroPedido} — lista todos los cambios de estado de un pedido.
- **GET** /api/estados/pedido/{numeroPedido}/ultimo — obtiene el último estado registrado de un pedido.

Comunica con: Es consumido vía Feign por pedido-service cada vez que se crea un pedido o cambia su estado.

Base de datos: Base H2 propia, tabla cambios_estado.

fabricacion-service (puerto 8085)

Responsabilidad: Gestiona las órdenes de fabricación asociadas a un pedido y su avance dentro del proceso productivo de la imprenta.

Entidades JPA:

- OrdenFabricacion: id, pedidold, estadoFabricacion (enum), fechaInicio, fechaFin, fechaCreacion, fechaActualizacion, descripcionEstado, usuarioResponsable; relación @OneToMany con HistorialFabricacion.
- HistorialFabricacion: id, ordenFabricacion (@ManyToOne), estadoAnterior, estadoNuevo, fechaCambio, usuariold, motivo.

Endpoints REST:

- **GET** /api/fabricacion/ping — healthcheck del servicio.
- **POST** /api/fabricacion — crea una orden de fabricación asociada a un pedido existente.
- **GET** /api/fabricacion/{id} — consulta una orden de fabricación por id.
- **PATCH** /api/fabricacion/{id}/estado — actualiza el estado de una orden de fabricación.

Comunica con: Llama vía Feign a pedido-service para validar el pedido asociado y notificar el cambio de estado de fabricación.

Base de datos: Base H2 propia, tablas ordenes_fabricacion e historial_fabricacion.

despacho-service (puerto 8084)

Responsabilidad: Administra el despacho de los pedidos ya fabricados, incluyendo el transportista asignado y el código de seguimiento.

Entidades JPA:

- Despacho: id, numeroPedido (único), tipoDespacho, transportista, fechaDespacho, trackingCode.

Endpoints REST:

- **POST** /api/despachos — registra un despacho asociado a un pedido existente.
- **GET** /api/despachos — lista todos los despachos, con filtro opcional por tipo.
- **GET** /api/despachos/{numeroPedido} — obtiene el despacho asociado a un pedido.
- **PUT** /api/despachos/{id} — actualiza los datos de un despacho.

Comunica con: Llama vía Feign a pedido-service para validar la existencia del pedido antes de registrar o actualizar su despacho.

Base de datos: Base H2 propia, tabla despachos con restricción de unicidad en numeroPedido.

transportista-service (puerto 8088)

Responsabilidad: Mantiene el registro de transportistas disponibles para realizar despachos y sus zonas de cobertura.

Entidades JPA:

- Transportista: id, nombre, codigoInterno, contacto, regionesCobertura, activo.

Endpoints REST:

- **POST** /api/transportistas — registra un nuevo transportista.
- **GET** /api/transportistas — lista los transportistas activos.
- **GET** /api/transportistas/{id} — obtiene un transportista por id.
- **PUT** /api/transportistas/{id} — actualiza los datos de un transportista.
- **GET** /api/transportistas/ping — healthcheck del servicio.

Comunica con: Servicio de soporte; no consume a otros servicios en la versión actual, pero está pensado para ser consultado por despacho-service al asignar transportista (próxima integración).

Base de datos: Base H2 propia, tabla transportistas.

metrica-service (puerto 8087)

Responsabilidad: Calcula y expone métricas analíticas del negocio: ranking de clientes, productos más vendidos y resumen de ventas por período.

Entidades JPA:

- **MetricaCliente:** clienteld, montoTotal, cantidadPedidos, frecuenciaAnual, ultimaActualizacion.
- **MetricaProducto:** productold, nombre, totalVendido, periodo, ultimaActualizacion.

Endpoints REST:

- **GET /api/metricas/clientes/{id}** — obtiene métricas agregadas de un cliente específico.
- **GET /api/metricas/clientes/ranking** — obtiene el ranking de clientes por monto total.
- **GET /api/metricas/productos/top** — obtiene los productos más vendidos en un rango de fechas.
- **GET /api/metricas/ventas** — obtiene un resumen de ventas en un rango de fechas.
- **GET /api/metricas/ping** — healthcheck del servicio.

Comunica con: Llama vía Feign a pedido-service (para obtener pedidos y calcular ventas) y a cliente-service (para enriquecer el ranking con datos del cliente).

Base de datos: Base H2 propia, tablas metricas_cliente y metricas_producto.

log-service (puerto 8089)

Responsabilidad: Centraliza el registro de logs de operaciones relevantes ocurridas en los distintos microservicios, permitiendo trazabilidad transversal del sistema.

Entidades JPA:

- **LogEntrada:** id, servicio, operacion, usuariold, timestamp, resultado, detalle.

Endpoints REST:

- **POST /api/logs** — registra una nueva entrada de log.
- **GET /api/logs** — consulta logs, con filtros opcionales por servicio y fecha.
- **GET /api/logs/ping** — healthcheck del servicio.

Comunica con: Servicio de soporte transversal; puede ser invocado por cualquier otro microservicio para dejar trazabilidad de sus operaciones.

Base de datos: Base H2 propia, tabla logs.

5.1 Resumen de comunicación entre servicios

Servicio origen → destino	Motivo de la comunicación
pedido-service → cliente-service	Validar que el cliente del pedido exista antes de crearlo.
pedido-service → producto-service	Validar productos y obtener su precio al crear un pedido.
pedido-service → estado-service	Registrar y consultar los cambios de estado del pedido.
despacho-service → pedido-service	Validar que el pedido a despachar exista.
fabricacion-service → pedido-service	Validar el pedido asociado y notificar cambios de estado de fabricación.
metrica-service → pedido-service	Obtener pedidos para calcular ventas y rankings de productos.
metrica-service → cliente-service	Enriquecer el ranking de clientes con sus datos.
api-gateway → auth-service	Validar el token JWT de cada petición entrante.

6. Checklist de entrega

- Identificación del problema con actores definidos.
- 27 requerimientos funcionales y 8 requerimientos no funcionales, numerados y verificables.
- Solución propuesta con diagrama de arquitectura.
- 12 historias de usuario con criterios de aceptación.
- 10 microservicios definidos con plantilla completa (nombre, responsabilidad, entidades, endpoints, dependencias, base de datos).
- Comunicación inter-servicio identificada en pedido-service, despacho-service, fabricacion-service y metrica-service.
- Documento incluido en el repositorio GitHub (README.md o PDF adjunto).